

HB using Annotation

To demonstrate a simple Hibernate example using annotations in Eclipse, we'll assume you have already set up the necessary environment with Hibernate and a compatible database driver. Here are the steps to create a basic Hibernate application:

Step 1: Set up the Maven Project

Step 2: Define the Entity Class

1. Create a new Java class called "Product" in your project.
2. Annotate the class with `@Entity` to mark it as a persistent entity.
3. Add the `@Table` annotation to specify the corresponding database table name.
4. Add the `@Id` annotation to denote the primary key field.

Example 1:

Product.java

```
package com.company.hibernateanno;

import javax.persistence.Entity;
import javax.persistence.Id;
import javax.persistence.Table;

@Entity
@Table(name = "productN")
public class Product {
    @Id
    private int Pid;
    private String Pname;
    private int Pprice;
    public Product() {

    }
    public Product(int pid, String pname, int pprice) {
        super();
        Pid = pid;
        Pname = pname;
        Pprice = pprice;
    }
    public int getPid() {
        return Pid;
    }
    public void setPid(int pid) {
        Pid = pid;
    }
}
```

```

    public String getPname() {
        return Pname;
    }
    public void setPname(String pname) {
        Pname = pname;
    }
    public int getPprice() {
        return Pprice;
    }
    public void setPprice(int pprice) {
        Pprice = pprice;
    }
    @Override
    public String toString() {
        return "Product [Pid=" + Pid + ", Pname=" + Pname + ",
Pprice=" + Pprice + "]";
    }
}

```

AppProduct.java

```

package com.company.hibernateanno;

import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;
import org.hibernate.cfg.Configuration;
import org.hibernate.service.ServiceRegistry;
import org.hibernate.service.ServiceRegistryBuilder;

public class AppProduct {
    static Session getSession() {
        Configuration config = new Configuration().configure()
            .addAnnotatedClass(Product.class);
        ServiceRegistry sr = new ServiceRegistryBuilder()
            .applySettings(config.getProperties())
            .buildServiceRegistry();
        SessionFactory factory = config.buildSessionFactory(sr);
        Session session = factory.openSession();
        return session;
    }
    public static void main(String[] args) {
        Product p1 = new Product();
        p1.setPid(103);
    }
}

```

```

        p1.setPname("Pencil");
        p1.setPprice(10);

        Session session = getSession();
        Transaction trans = session.beginTransaction();
        session.save(p1);
        trans.commit();
    }
}

```

hibernate.cfg.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-configuration PUBLIC
    "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
    "http://www.hibernate.org/dtd/hibernate-configuration-
3.0.dtd">
<hibernate-configuration>
    <session-factory>
        <property
name="hibernate.connection.driver_class">oracle.jdbc.driver.OracleDriv
er</property>
        <property
name="hibernate.connection.password">oracle</property>
        <property
name="hibernate.connection.url">jdbc:oracle:thin:@localhost:1521:xe</p
roperty>
        <property
name="hibernate.connection.username">system</property>
        <property
name="hibernate.dialect">org.hibernate.dialect.Oracle10gDialect</prope
rty>
        <property name="hbm2ddl.auto">create</property>
        <property name="show_sql">>true</property>
    </session-factory>
</hibernate-configuration>

```

pom.xml

```

<dependencies>
<!-- https://mvnrepository.com/artifact/org.hibernate/hibernate-core -->
<dependency>
    <groupId>org.hibernate</groupId>
    <artifactId>hibernate-core</artifactId>
    <version>4.1.6.Final</version>

```

```
</dependency>
```

```
<!-- https://mvnrepository.com/artifact/com.oracle.database.jdbc/ojdbc10-->
<dependency>
    <groupId>com.oracle.database.jdbc</groupId>
    <artifactId>ojdbc10</artifactId>
    <version>19.16.0.0</version>
</dependency>
</dependencies>
</project>
```

Run the application

To run the hibernate application; right click on the AppProduct class - Run As - Java Application.

Example 2:

Here's an example of the "Student" class:

Student.java

```
package com.company.hibernateanno;

import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import javax.persistence.Table;

@Entity
@Table(name = "students")
public class Student {
    @Id
    @GeneratedValue(strategy = GenerationType.TABLE)
    private int id;

    @Column(name = "name")
    private String name;

    @Column(name = "age")
    private int age;

    // Constructors, getters, and
```

```

public Student() {
    super();
}

public Student(int id, String name, int age) {
    super();
    this.id = id;
    this.name = name;
    this.age = age;
}

public int getId() {
    return id;
}

public String getName() {
    return name;
}

public int getAge() {
    return age;
}

public void setId(int id) {
    this.id = id;
}

public void setName(String name) {
    this.name = name;
}

public void setAge(int age) {
    this.age = age;
}
}

```

Step 2: Configure Hibernate

1. Create a Hibernate configuration file (e.g., `hibernate.cfg.xml`) in your project's classpath (right click on **src/main/java** - **new** - **file** - specify the file name (e.g. hibernate.cfg.xml) - **Finish**).
2. Configure the database connection details, dialect, and other properties in the configuration file. For example:

hibernate.cfg.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-configuration PUBLIC
    "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
    "http://www.hibernate.org/dtd/hibernate-
configuration-3.0.dtd">
<hibernate-configuration>
    <session-factory>
        <property
name="hibernate.current_session_context_class">thread</prope
rty>
            <property
name="hibernate.connection.driver_class">oracle.jdbc.driver.
OracleDriver</property>
            <property
name="hibernate.connection.password">oracle</property>
            <property
name="hibernate.connection.url">jdbc:oracle:thin:@localhost:
1521:xe</property>
            <property
name="hibernate.connection.username">system</property>
            <property
name="hibernate.dialect">org.hibernate.dialect.Oracle10gDial
ect</property>
            <property name="hbm2ddl.auto">update</property>
            <property name="show_sql">true</property>

            <mapping class="com.company.hibernateanno.Student"/>
        </session-factory>
    </hibernate-configuration>
```

Step 3: Create Hibernate Util Class

1. Create a utility class to manage the Hibernate SessionFactory and Session objects.
2. Implement methods to initialize the SessionFactory and retrieve the current Session.

HibernateUtil.java

```
package com.company.hibernateanno;
```

```

import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.cfg.Configuration;
import org.hibernate.service.ServiceRegistry;
import org.hibernate.service.ServiceRegistryBuilder;

public class HibernateUtil {
    private static final SessionFactory sessionFactory =
buildSessionFactory();

    private static SessionFactory buildSessionFactory() {
        try {
            Configuration config = new Configuration().configure();
            ServiceRegistry sr = new ServiceRegistryBuilder()
                .applySettings(config.getProperties())
                .buildServiceRegistry();
            SessionFactory factory = config.buildSessionFactory(sr);
            return factory;
        } catch (Throwable ex) {
            System.err.println("Initial SessionFactory creation
failed: " + ex);
            throw new ExceptionInInitializerError(ex);
        }
    }

    public static SessionFactory getSessionFactory() {
        return sessionFactory;
    }

    public static Session getCurrentSession() {
        return sessionFactory.getCurrentSession();
    }
}

```

pom.xml

```

<dependencies>
<!-- https://mvnrepository.com/artifact/org.hibernate/hibernate-core -->
<dependency>
    <groupId>org.hibernate</groupId>
    <artifactId>hibernate-core</artifactId>
    <version>4.1.6.Final</version>
</dependency>

<!-- https://mvnrepository.com/artifact/com.oracle.database.jdbc/ojdbc10-->

```

```
<dependency>
  <groupId>com.oracle.database.jdbc</groupId>
  <artifactId>ojdbc10</artifactId>
  <version>19.16.0.0</version>
</dependency>
</dependencies>
</project>
```

Step 4: Perform Database Operations

1. In your main class 'AppStudent' or any other suitable class, initialize the Hibernate SessionFactory.
2. Use the Session object to perform CRUD operations or execute queries.

Here's an example of creating a new student and saving it to the database:

AppStudent.java

```
package com.company.hibernateanno;

import org.hibernate.Session;
import org.hibernate.SessionFactory;

public class AppStudent {
    public static void main(String[] args) {
        // Initialize the Hibernate SessionFactory
        SessionFactory sessionFactory =
        HibernateUtil.getSessionFactory();

        // Get the current session
        Session session = HibernateUtil.getCurrentSession();

        try {
            // Create a new student
            Student student = new Student();
            student.setName("John Doe");
            student.setAge(20);

            // Start a transaction
            session.beginTransaction();

            // Save the student to the database
            session.save(student);

            // Commit the transaction
```

```

        session.getTransaction().commit();

        System.out.println("Student saved successfully!");
    } catch (Exception e) {
        // Handle any exceptions
        e.printStackTrace();
        session.getTransaction().rollback();
    } finally {
        // Close the session and the SessionFactory
        session.close();
        sessionFactory.close();
    }
}
}

```

To run the hibernate application, right click on the AppStudent - Run As - Java Application.

That's it! We've created a simple Hibernate application using annotations in Eclipse.

Additional Tasks:

Expand this example by adding more entities, performing additional database operations, and exploring Hibernate's features like querying and relationships.

Remember to handle exceptions appropriately and close the resources to ensure proper cleanup.